고객을 세그먼테이션하자 [프로젝트] - 박지민

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT COUNT(*) AS total_rows
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

```
SELECT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
LIMIT 10;
```

실행 시 이 쿼리가 46.02MB를 처리합니다.

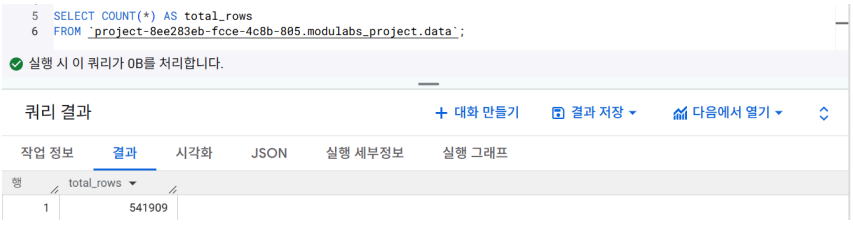
쿼리 결과 | + 대화 만들기 | 결과 저장 | 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate
1	536365	85123A	WHITE HANGING HEART T-LIG...	6	2010-12-01 08:26
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26
7	536365	21730	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28
10	536367	84879	ASSORTED COLOUR BIRD ORN...	32	2010-12-01 08:34

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*) AS total_rows
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```



```
SELECT COUNT(*) AS total_rows
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

실행 시 이 쿼리가 0B를 처리합니다.

쿼리 결과 | + 대화 만들기 | 결과 저장 | 다음에서 열기

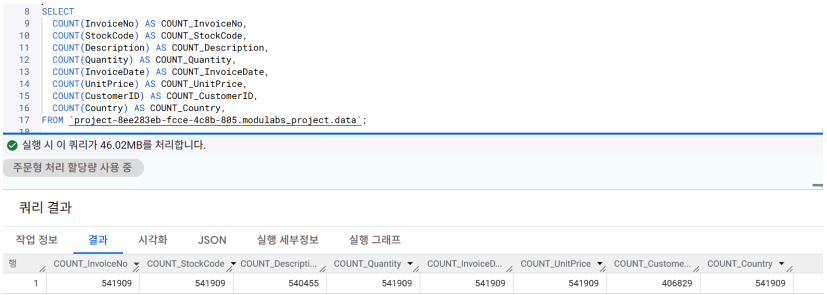
작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	total_rows
1	541909

데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT
  COUNT(InvoiceNo) AS COUNT_InvoiceNo,
  COUNT(StockCode) AS COUNT_StockCode,
  COUNT(Description) AS COUNT_Description,
  COUNT(Quantity) AS COUNT_Quantity,
  COUNT(InvoiceDate) AS COUNT_InvoiceDate,
  COUNT(UnitPrice) AS COUNT_UnitPrice,
  COUNT(CustomerID) AS COUNT_CustomerID,
  COUNT(Country) AS COUNT_Country,
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```



```
SELECT
  COUNT(InvoiceNo) AS COUNT_InvoiceNo,
  COUNT(StockCode) AS COUNT_StockCode,
  COUNT(Description) AS COUNT_Description,
  COUNT(Quantity) AS COUNT_Quantity,
  COUNT(InvoiceDate) AS COUNT_InvoiceDate,
  COUNT(UnitPrice) AS COUNT_UnitPrice,
  COUNT(CustomerID) AS COUNT_CustomerID,
  COUNT(Country) AS COUNT_Country,
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

실행 시 이 쿼리가 46.02MB를 처리합니다.

주문형 처리 할당량 사용 중

쿼리 결과

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Descripti...	COUNT_Quantity	COUNT_InvoiceD...	COUNT_UnitPrice	COUNT_Custome...	COUNT_Country
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT
    'InvoiceNo' AS column_name,
    ROUND(SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS missing_perce
ntage
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`

UNION ALL

SELECT
    'StockCode',
    ROUND(SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`

UNION ALL

SELECT
    'Description',
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`

UNION ALL

SELECT
    'Quantity',
    ROUND(SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`

UNION ALL

SELECT
    'InvoiceDate',
    ROUND(SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`

UNION ALL

SELECT
    'UnitPrice',
    ROUND(SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`

UNION ALL

SELECT
```

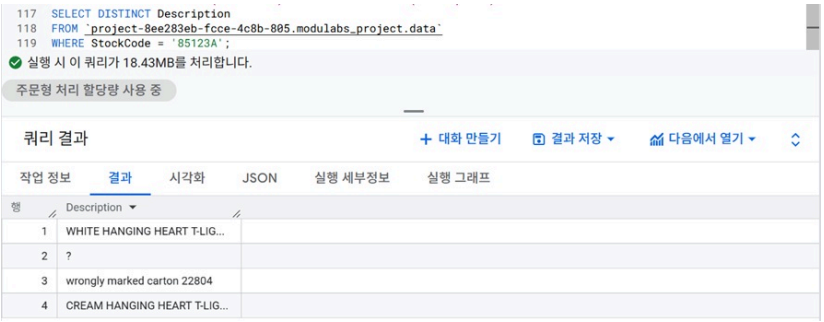
```
    'CustomerID',
    ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`

UNION ALL

SELECT
    'Country',
    ROUND(SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100, 2)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

행	column_name	missing_percenta...
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```
SELECT DISTINCT Description
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE StockCode = '85123A';
```



행	Description
1	WHITE HANGING HEART T-LIG...
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIG...

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

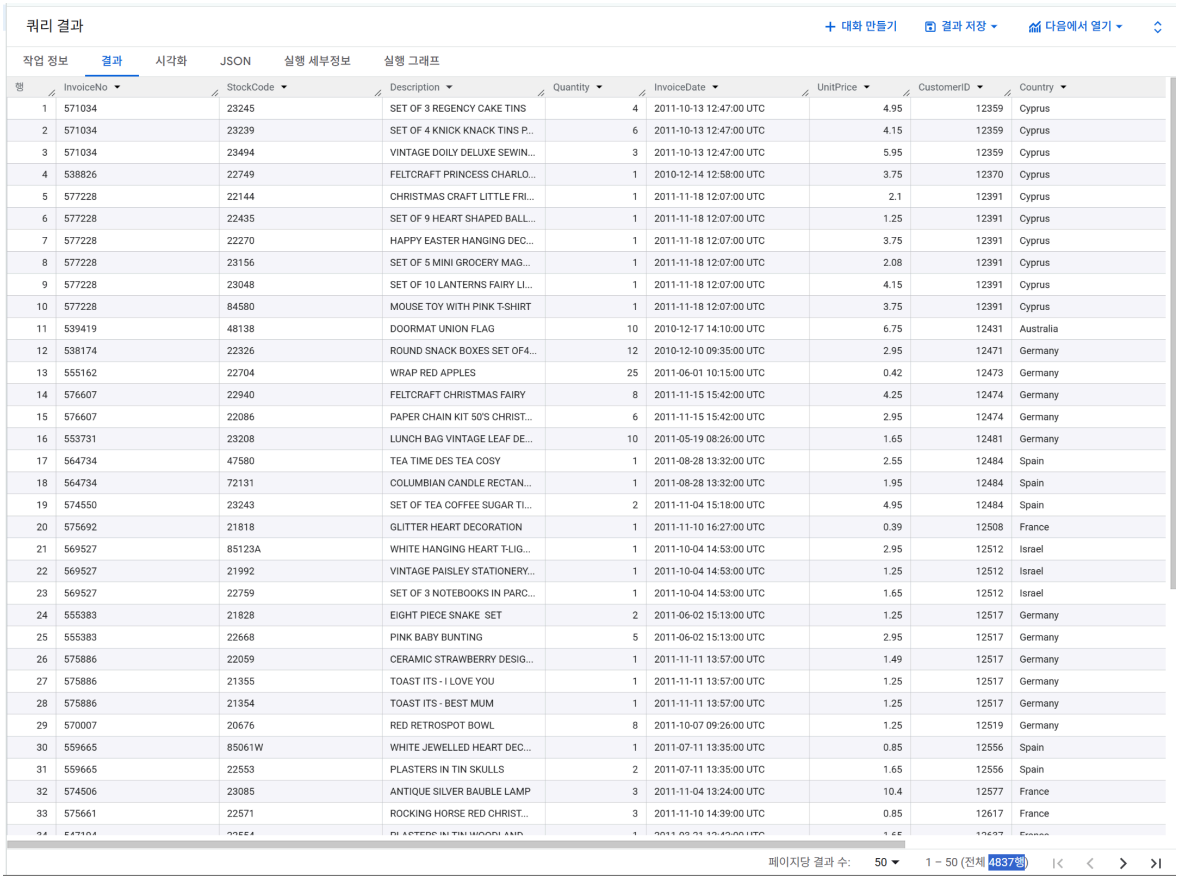
```
DELETE FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE Description IS NULL
   OR CustomerID IS NULL;
```

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
SELECT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY
    InvoiceNo,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
    Country
HAVING COUNT(*) > 1;
```

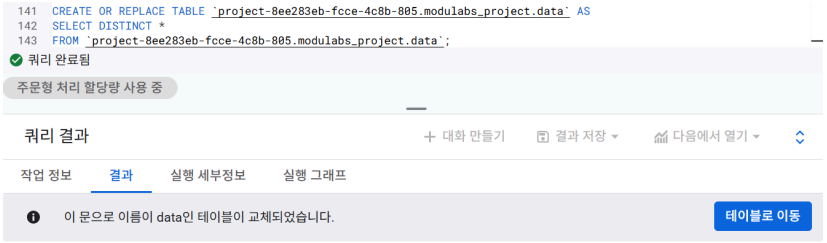


행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	571034	23245	SET OF 3 REGENCY CAKE TINS	4	2011-10-13 12:47:00 UTC	4.95	12359	Cyprus
2	571034	23239	SET OF 4 KNICK KNACK TINS P...	6	2011-10-13 12:47:00 UTC	4.15	12359	Cyprus
3	571034	23494	VINTAGE DOILY DELUXE SEWIN...	3	2011-10-13 12:47:00 UTC	5.95	12359	Cyprus
4	538826	22749	FELTCRAFT PRINCESS CHARLO...	1	2010-12-14 12:58:00 UTC	3.75	12370	Cyprus
5	577228	22144	CHRISTMAS CRAFT LITTLE FRI...	1	2011-11-18 12:07:00 UTC	2.1	12391	Cyprus
6	577228	22435	SET OF 9 HEART SHAPED BALL...	1	2011-11-18 12:07:00 UTC	1.25	12391	Cyprus
7	577228	22270	HAPPY EASTER HANGING DEC...	1	2011-11-18 12:07:00 UTC	3.75	12391	Cyprus
8	577228	23156	SET OF 5 MINI GROCERY MAG...	1	2011-11-18 12:07:00 UTC	2.08	12391	Cyprus
9	577228	23048	SET OF 10 LANTERNS FAIRY LI...	1	2011-11-18 12:07:00 UTC	4.15	12391	Cyprus
10	577228	84580	MOUSE TOY WITH PINK T-SHIRT	1	2011-11-18 12:07:00 UTC	3.75	12391	Cyprus
11	539419	48138	DOORMAT UNION FLAG	10	2010-12-17 14:10:00 UTC	6.75	12431	Australia
12	538174	22326	ROUND SNACK BOXES SET OF4...	12	2010-12-10 09:35:00 UTC	2.95	12471	Germany
13	555162	22704	WRAP RED APPLES	25	2011-06-01 10:15:00 UTC	0.42	12473	Germany
14	576607	22940	FELTCRAFT CHRISTMAS FAIRY	8	2011-11-15 15:42:00 UTC	4.25	12474	Germany
15	576607	22086	PAPER CHAIN KIT 50'S CHRIST...	6	2011-11-15 15:42:00 UTC	2.95	12474	Germany
16	553731	23208	LUNCH BAG VINTAGE LEAF DE...	10	2011-05-19 08:26:00 UTC	1.65	12481	Germany
17	564734	47580	TEA TIME DES TEA COSY	1	2011-08-28 13:32:00 UTC	2.55	12484	Spain
18	564734	72131	COLUMBIAN CANDLE RECTAN...	1	2011-08-28 13:32:00 UTC	1.95	12484	Spain
19	574550	23243	SET OF TEA COFFEE SUGAR TI...	2	2011-11-04 15:18:00 UTC	4.95	12484	Spain
20	575692	21818	GLITTER HEART DECORATION	1	2011-11-10 16:27:00 UTC	0.39	12508	France
21	569527	85123A	WHITE HANGING HEART T-LIG...	1	2011-10-04 14:53:00 UTC	2.95	12512	Israel
22	569527	21992	VINTAGE PAISLEY STATIONERY...	1	2011-10-04 14:53:00 UTC	1.25	12512	Israel
23	569527	22759	SET OF 3 NOTEBOOKS IN PARC...	1	2011-10-04 14:53:00 UTC	1.65	12512	Israel
24	555383	21828	EIGHT PIECE SNAKE SET	2	2011-06-02 15:13:00 UTC	1.25	12517	Germany
25	555383	22668	PINK BABY BUNTING	5	2011-06-02 15:13:00 UTC	2.95	12517	Germany
26	575886	22059	CERAMIC STRAWBERRY DESIG...	1	2011-11-11 13:57:00 UTC	1.49	12517	Germany
27	575886	21355	TOAST ITS - I LOVE YOU	1	2011-11-11 13:57:00 UTC	1.25	12517	Germany
28	575886	21354	TOAST ITS - BEST MUM	1	2011-11-11 13:57:00 UTC	1.25	12517	Germany
29	570007	20676	RED RETROSPOT BOWL	8	2011-10-07 09:26:00 UTC	1.25	12519	Germany
30	559665	85061W	WHITE JEWELLED HEART DEC...	1	2011-07-11 13:35:00 UTC	0.85	12556	Spain
31	559665	22553	PLASTERS IN TIN SKULLS	2	2011-07-11 13:35:00 UTC	1.65	12556	Spain
32	574506	23085	ANTIQUE SILVER BAUBLE LAMP	3	2011-11-04 13:24:00 UTC	10.4	12577	France
33	575661	22571	ROCKING HORSE RED CHRIST...	3	2011-11-10 14:39:00 UTC	0.85	12617	France

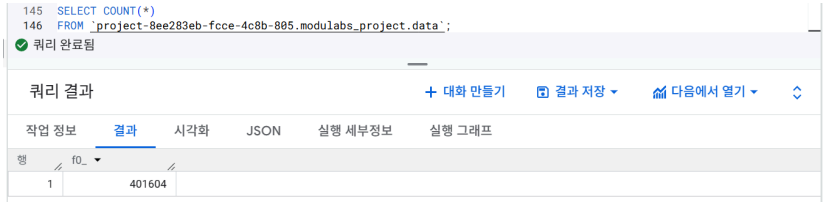
중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.data` AS
SELECT DISTINCT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```



cf. 중복값을 처리하고 난 후 남은 데이터 행의 개수

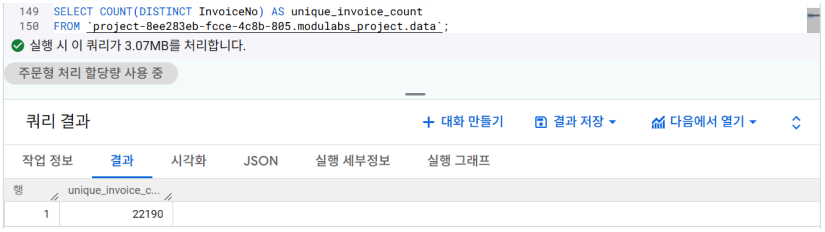


11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

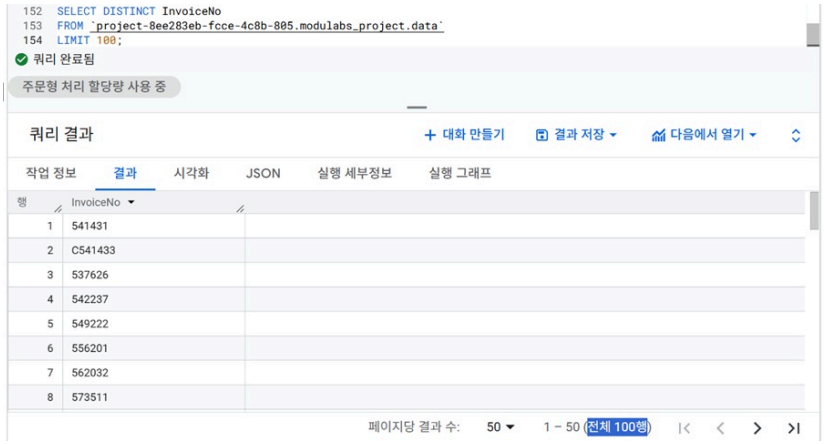
- **고유(unique)한 `InvoiceNo` 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoice_count
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```



- **고유한 `InvoiceNo` 를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
LIMIT 100;
```



```
SELECT DISTINCT InvoiceNo
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
LIMIT 100;
```

쿼리 완료됨

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

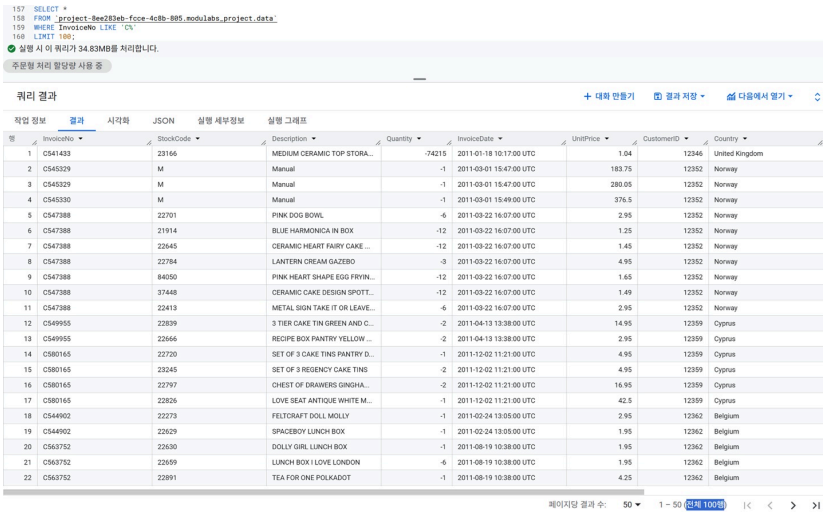
작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511

페이지당 결과 수: 50 1 – 50 (전체 100행)

- `InvoiceNo` 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```



```
SELECT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

실행 시 이 쿼리가 34.83MB를 처리합니다.

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

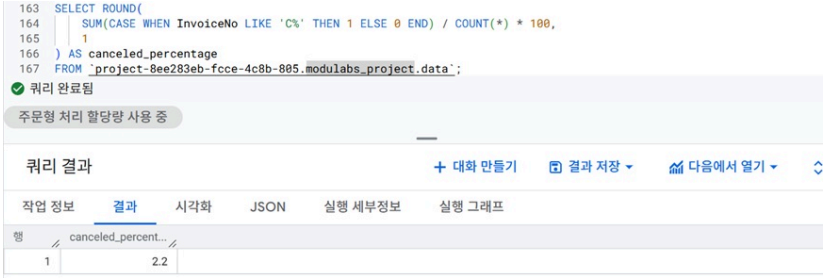
작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
6	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	1.25	12352	Norway
7	C547388	22645	CERAMIC HEART FAIRY CAKE ...	-12	2011-03-22 16:07:00 UTC	1.45	12352	Norway
8	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
9	C547388	84050	PINK HEART SHAPE EGG FRYIN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
10	C547388	37448	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	1.49	12352	Norway
11	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
12	C549955	22839	3 TIER CAKE TIN GREEN AND C...	-2	2011-04-13 13:38:00 UTC	14.95	12359	Cyprus
13	C549955	22666	RECIPE BOX PANTRY YELLOW ...	-2	2011-04-13 13:38:00 UTC	2.95	12359	Cyprus
14	C580165	22720	SET OF 3 CAKE TINS PANTRY D...	-1	2011-12-02 11:21:00 UTC	4.95	12359	Cyprus
15	C580165	23245	SET OF 3 REGENCY CAKE TINS	-2	2011-12-02 11:21:00 UTC	4.95	12359	Cyprus
16	C580165	22797	CHEST OF DRAWERS GINGHA...	-2	2011-12-02 11:21:00 UTC	16.95	12359	Cyprus
17	C580165	22826	LOVE SEAT ANTIQUE WHITE M...	-1	2011-12-02 11:21:00 UTC	42.5	12359	Cyprus
18	C544902	22273	FELTCRAFT DOLL MOLLY	-1	2011-02-24 13:05:00 UTC	2.95	12362	Belgium
19	C544902	22629	SPACEBOY LUNCH BOX	-1	2011-02-24 13:05:00 UTC	1.95	12362	Belgium
20	C563752	22630	DOLLY GIRL LUNCH BOX	-1	2011-08-19 10:38:00 UTC	1.95	12362	Belgium
21	C563752	22659	LUNCH BOX I LOVE LONDON	-6	2011-08-19 10:38:00 UTC	1.95	12362	Belgium
22	C563752	22891	TEA FOR ONE POLKADOT	-1	2011-08-19 10:38:00 UTC	4.25	12362	Belgium

페이지당 결과 수: 50 1 – 50 (전체 100행)

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT ROUND(
    SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*) * 100,
    1
) AS canceled_percentage
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```



```
SELECT ROUND(
    SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*) * 100,
    1
) AS canceled_percentage
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

쿼리 완료됨

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

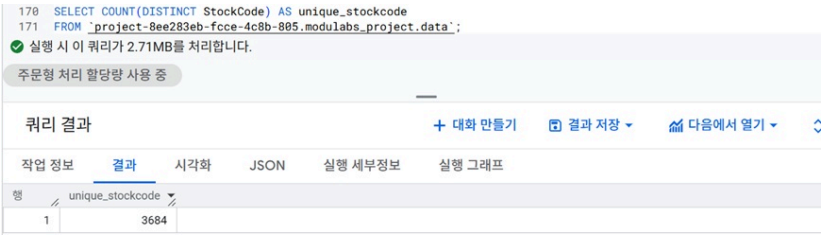
작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	canceled_percent...
1	2.2

`StockCode` 살펴보기

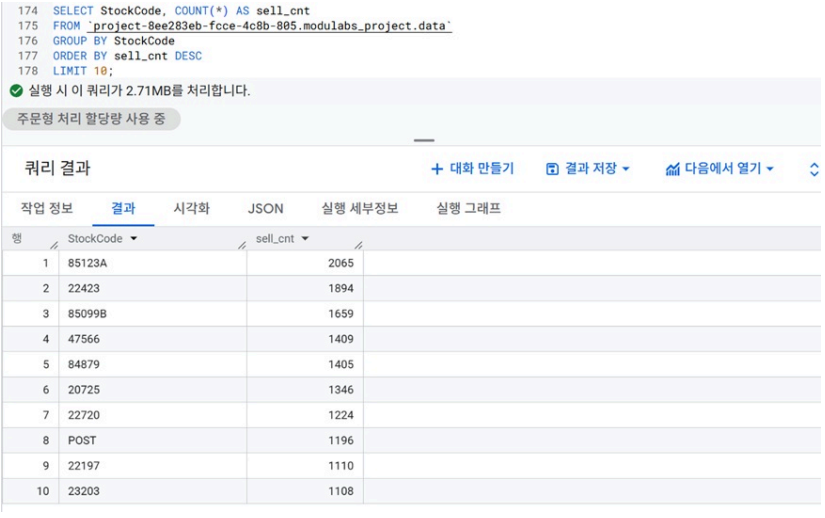
- **고유한 `StockCode` 의 개수를 출력하기**

```
SELECT COUNT(DISTINCT StockCode) AS unique_stockcode
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```



- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```



- **`StockCode` 의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
)
WHERE number_count <= 1;
```

```sql
SELECT DISTINCT StockCode, number_count
FROM (
    SELECT StockCode,
           LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
)
WHERE number_count <= 1;
```

쿼리 완료됨

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```sql
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM project_name.modulabs_project.data
)
WHERE # [[YOUR QUERY]];
```

```sql
SELECT ROUND(
    SUM(CASE WHEN number_count <= 1 THEN 1 ELSE 0 END) / COUNT(*) * 100,
    2
) AS abnormal_percentage
FROM (
    SELECT LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
);
```

쿼리 완료됨

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	abnormal_percen...
1	0.48

- **제품과 관련되지 않은 거래 기록을 제거하기**

```sql
DELETE FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE StockCode IN (
  SELECT DISTINCT StockCode
    FROM (
        SELECT StockCode,
               LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
        FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
    )
    WHERE number_count <= 1
);
```

```sql
DELETE FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE StockCode IN (
    SELECT DISTINCT StockCode
    FROM (
        SELECT StockCode,
            LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
        FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
    )
    WHERE number_count <= 1
);
```

실행 시 이 쿼리가 34.71MB를 처리합니다.

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 data의 행 1,915개가 삭제되었습니다. 테이블로 이동

`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```sql
SELECT Description, COUNT(*) AS description_cnt
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

```sql
SELECT Description, COUNT(*) AS description_cnt
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

쿼리 완료됨

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 시각화 JSON 실행 세부정보 실행 그래프

행	Description	description_cnt
1	WHITE HANGING HEART T-LIG...	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORN...	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY D...	1224
8	LUNCH BAG BLACK SKULL.	1099
9	PACK OF 72 RETROSPOT CAKE ...	1062
10	SPOTTY BUNTING	1026
11	PAPER CHAIN KIT 50'S CHRIST...	1013
12	LUNCH BAG SPACEBOY DESIGN	1006
13	LUNCH BAG CARS BLUE	1000

페이지당 결과 수: 50 1 – 30 (전체 30행)

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```sql
DELETE
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE Description IN ('Next Day Carriage', 'High Resolution Image');
```

```sql
DELETE
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE Description IN ('Next Day Carriage', 'High Resolution Image');
```

실행 시 이 쿼리가 34.7MB를 처리합니다.

주문형 처리 할당량 사용 중

쿼리 결과 + 대화 만들기 결과 저장 다음에서 열기

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 data의 행 83개가 삭제되었습니다. 테이블로 이동

- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.data` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

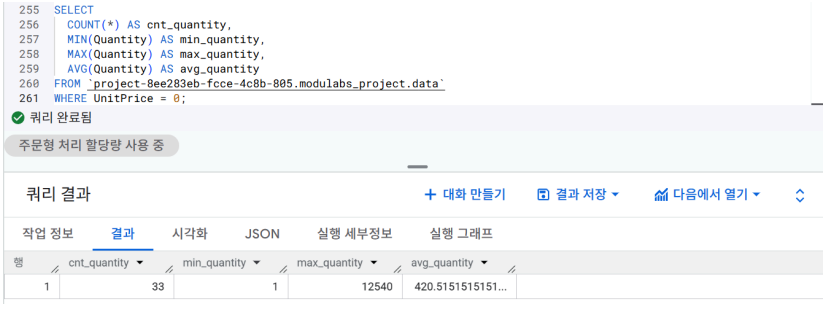
`UnitPrice` 살펴보기

- **`UnitPrice`의 최솟값, 최댓값, 평균을 구하기**

```
SELECT
  MIN(UnitPrice) AS min_price,
  MAX(UnitPrice) AS max_price,
  AVG(UnitPrice) AS avg_price
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

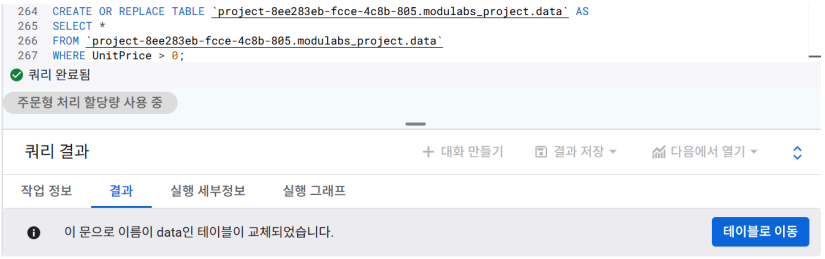
- **단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기**

```
SELECT
  COUNT(*) AS cnt_quantity,
  MIN(Quantity) AS min_quantity,
  MAX(Quantity) AS max_quantity,
  AVG(Quantity) AS avg_quantity
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE UnitPrice = 0;
```



- `UnitPrice = 0` 를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.data` AS
SELECT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
WHERE UnitPrice > 0;
```

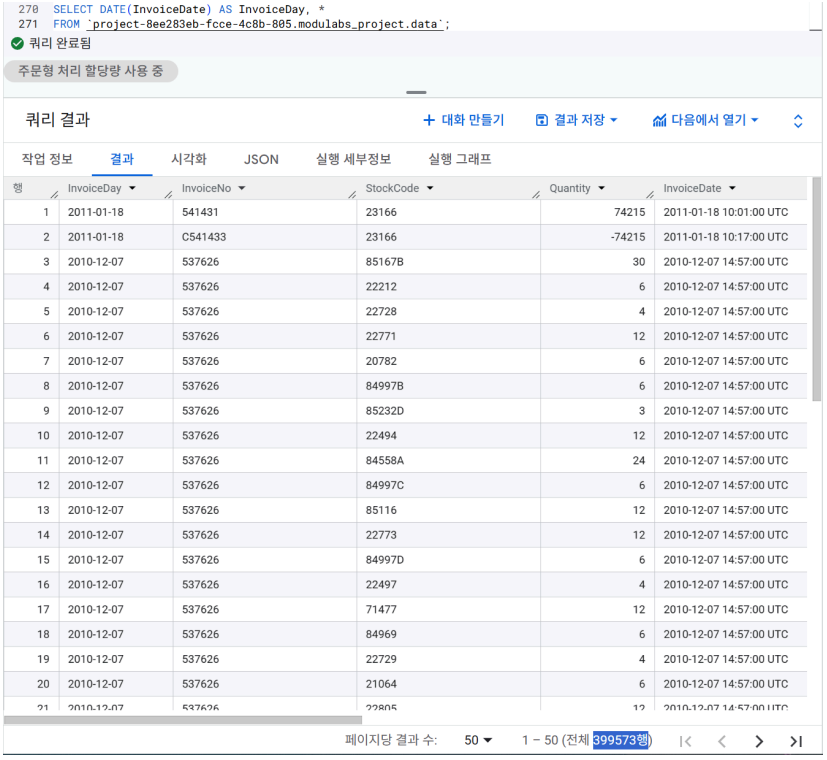


11-7. RFM 스코어

Recency

- `InvoiceDate` 컬럼을 연월일 자료형으로 변경하기

```
SELECT DATE(InvoiceDate) AS InvoiceDay, *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```

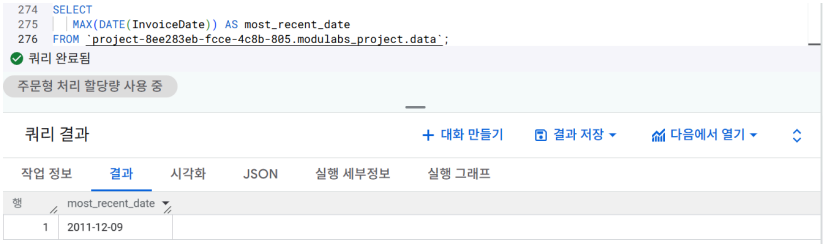


행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC
3	2010-12-07	537626	85167B	30	2010-12-07 14:57:00 UTC
4	2010-12-07	537626	22212	6	2010-12-07 14:57:00 UTC
5	2010-12-07	537626	22728	4	2010-12-07 14:57:00 UTC
6	2010-12-07	537626	22771	12	2010-12-07 14:57:00 UTC
7	2010-12-07	537626	20782	6	2010-12-07 14:57:00 UTC
8	2010-12-07	537626	84997B	6	2010-12-07 14:57:00 UTC
9	2010-12-07	537626	85232D	3	2010-12-07 14:57:00 UTC
10	2010-12-07	537626	22494	12	2010-12-07 14:57:00 UTC
11	2010-12-07	537626	84558A	24	2010-12-07 14:57:00 UTC
12	2010-12-07	537626	84997C	6	2010-12-07 14:57:00 UTC
13	2010-12-07	537626	85116	12	2010-12-07 14:57:00 UTC
14	2010-12-07	537626	22773	12	2010-12-07 14:57:00 UTC
15	2010-12-07	537626	84997D	6	2010-12-07 14:57:00 UTC
16	2010-12-07	537626	22497	4	2010-12-07 14:57:00 UTC
17	2010-12-07	537626	71477	12	2010-12-07 14:57:00 UTC
18	2010-12-07	537626	84969	6	2010-12-07 14:57:00 UTC
19	2010-12-07	537626	22729	4	2010-12-07 14:57:00 UTC
20	2010-12-07	537626	21064	6	2010-12-07 14:57:00 UTC
21	2010-12-07	537626	22805	12	2010-12-07 14:57:00 UTC

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

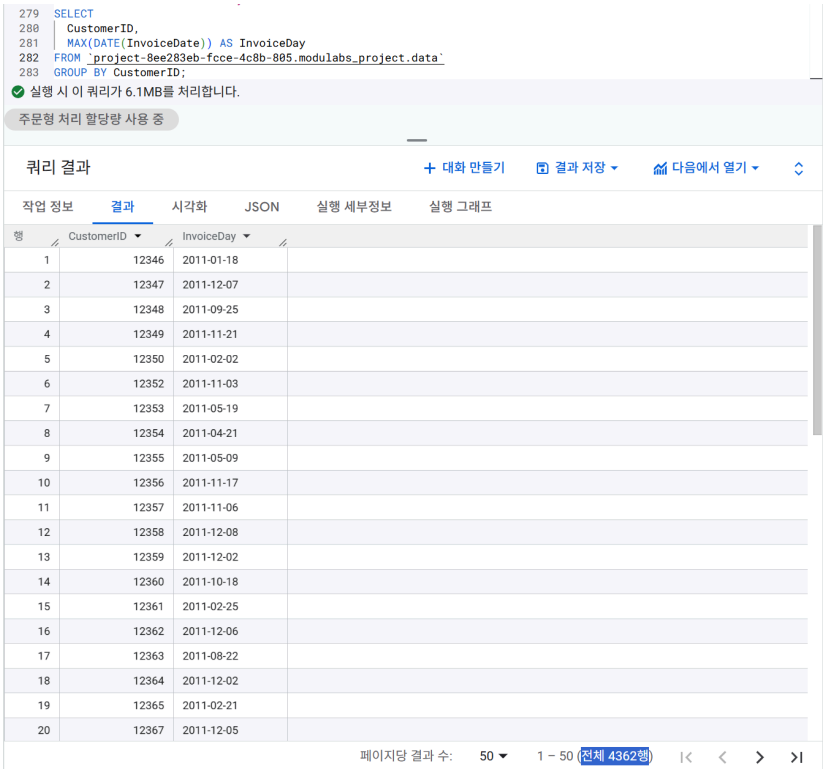
```
SELECT
   MAX(DATE(InvoiceDate)) AS most_recent_date
```

```
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`;
```



- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
  CustomerID,
  MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY CustomerID;
```



- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
  GROUP BY CustomerID
);
```

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

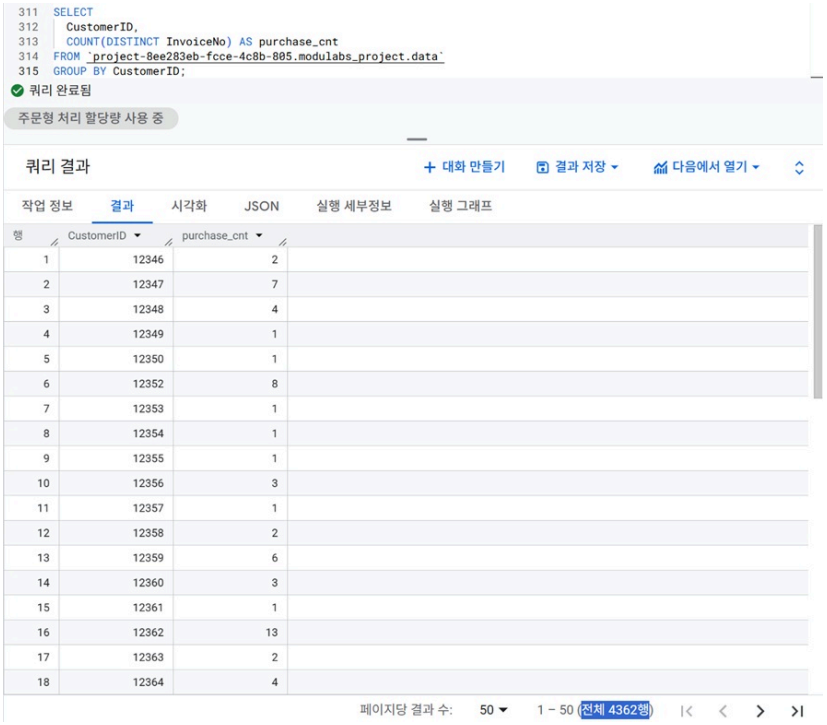
```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_r` AS
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
  GROUP BY CustomerID
);
```

Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

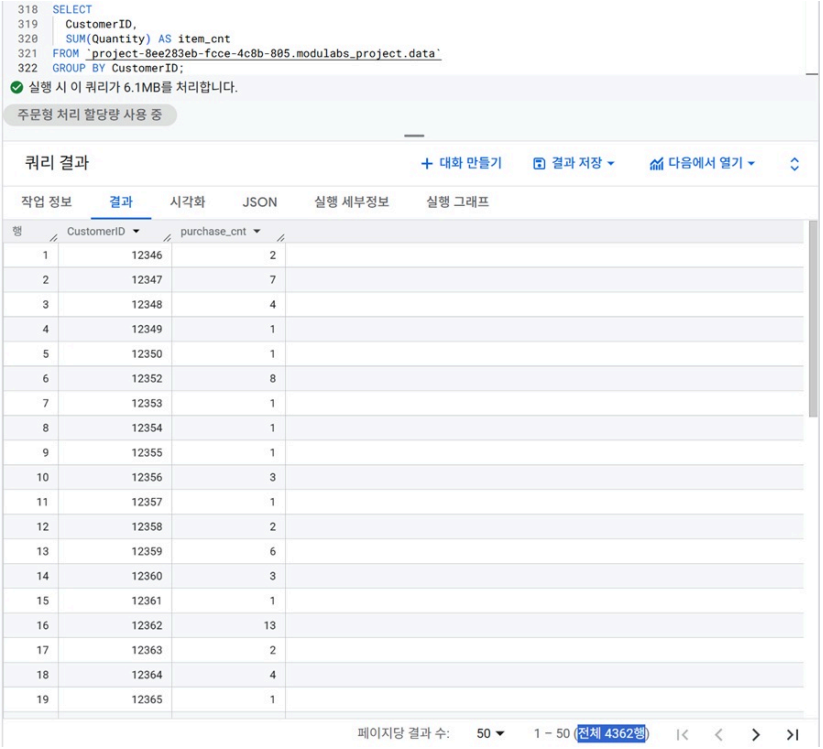
```
SELECT
  CustomerID,
```

```
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY CustomerID;
```



- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY CustomerID;
```



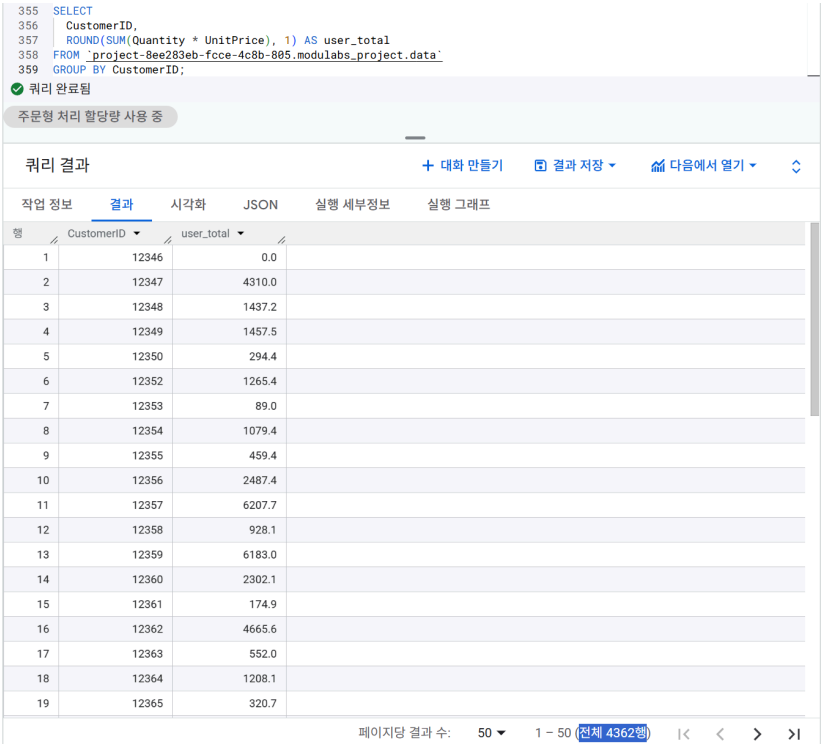
- **전체 거래 건수 계산와 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_rf` AS

WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
  GROUP BY CustomerID
),

item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
  GROUP BY CustomerID
)

SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt pc
JOIN item_cnt ic
  ON pc.CustomerID = ic.CustomerID
JOIN `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_r` ur
  ON pc.CustomerID = ur.CustomerID;
```

Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
GROUP BY CustomerID;
```



- **고객별 평균 거래 금액 계산**

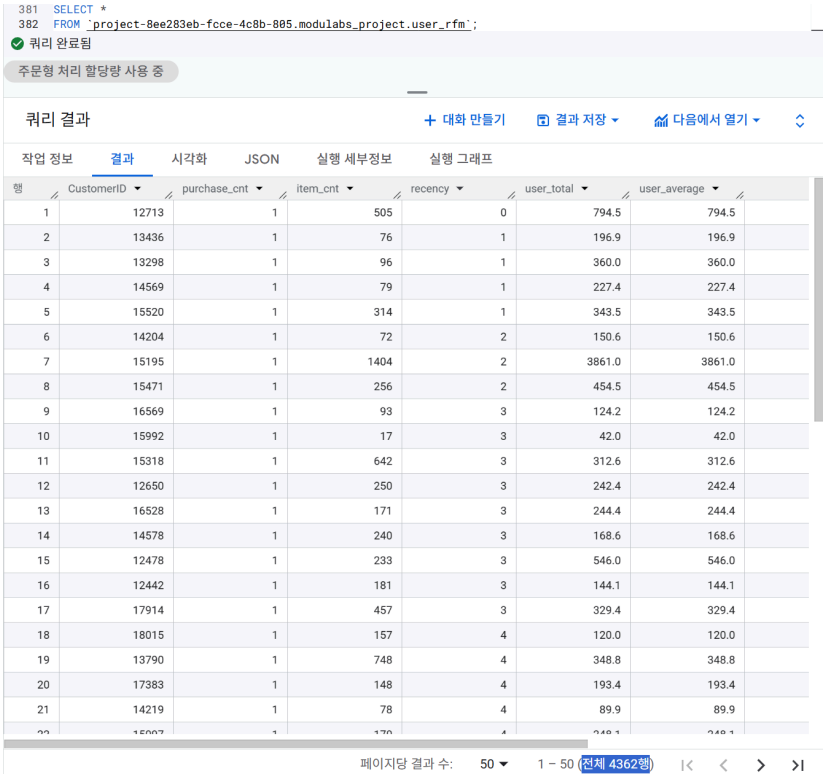
- **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_average
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_rf` rf
LEFT JOIN (
  SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
  GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;
```

RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_rfm`;
```



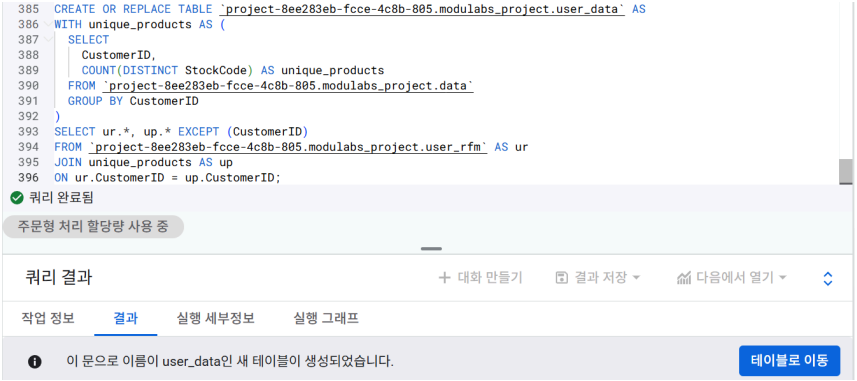
행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	13436	1	76	1	196.9	196.9
3	13298	1	96	1	360.0	360.0
4	14569	1	79	1	227.4	227.4
5	15520	1	314	1	343.5	343.5
6	14204	1	72	2	150.6	150.6
7	15195	1	1404	2	3861.0	3861.0
8	15471	1	256	2	454.5	454.5
9	16569	1	93	3	124.2	124.2
10	15992	1	17	3	42.0	42.0
11	15318	1	642	3	312.6	312.6
12	12650	1	250	3	242.4	242.4
13	16528	1	171	3	244.4	244.4
14	14578	1	240	3	168.6	168.6
15	12478	1	233	3	546.0	546.0
16	12442	1	181	3	144.1	144.1
17	17914	1	457	3	329.4	329.4
18	18015	1	157	4	120.0	120.0
19	13790	1	748	4	348.8	348.8
20	17383	1	148	4	193.4	193.4
21	14219	1	78	4	89.9	89.9

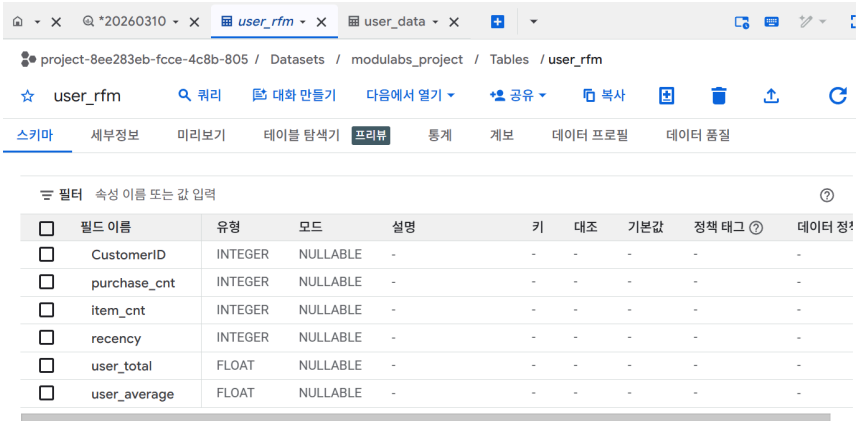
11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

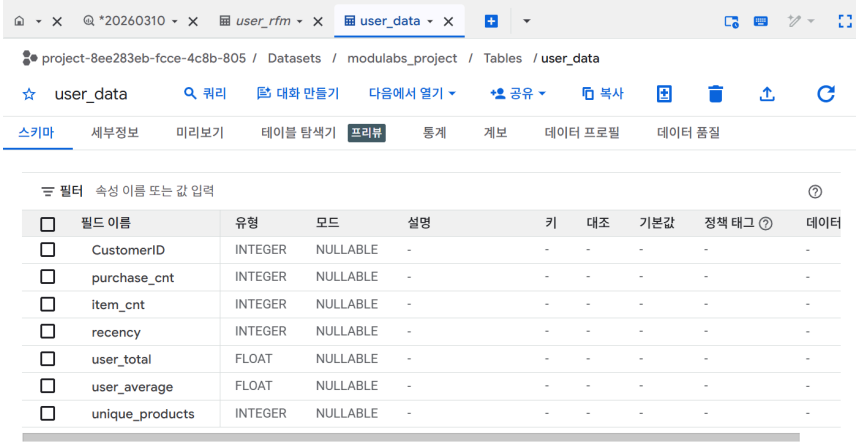
- **1) 고객 별로 구매한 상품들의 고유한 수를 계산하기**
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_data` AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
  GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```





필드 이름	유형	모드	설명	키	대조	기본값	정책 태그	데이터 정
CustomerID	INTEGER	NULLABLE	-	-	-	-	-	-
purchase_cnt	INTEGER	NULLABLE	-	-	-	-	-	-
item_cnt	INTEGER	NULLABLE	-	-	-	-	-	-
recency	INTEGER	NULLABLE	-	-	-	-	-	-
user_total	FLOAT	NULLABLE	-	-	-	-	-	-
user_average	FLOAT	NULLABLE	-	-	-	-	-	-



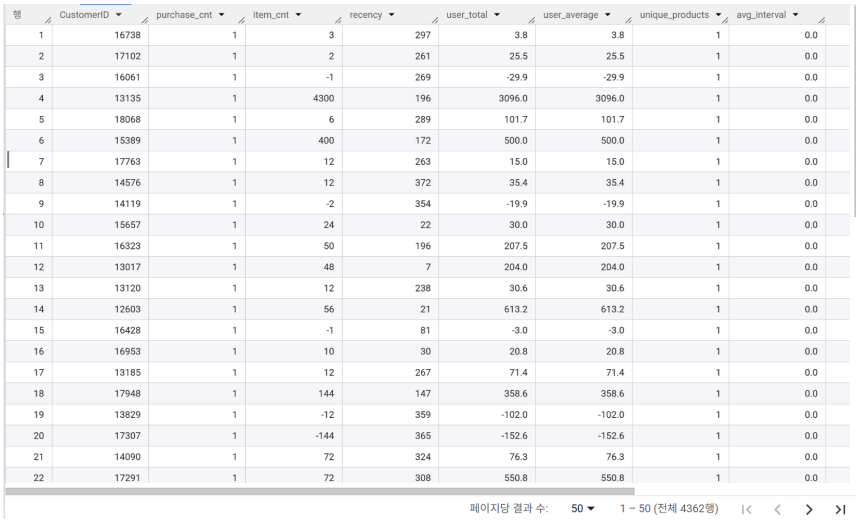
필드 이름	유형	모드	설명	키	대조	기본값	정책 태그	데이터
CustomerID	INTEGER	NULLABLE	-	-	-	-	-	-
purchase_cnt	INTEGER	NULLABLE	-	-	-	-	-	-
item_cnt	INTEGER	NULLABLE	-	-	-	-	-	-
recency	INTEGER	NULLABLE	-	-	-	-	-	-
user_total	FLOAT	NULLABLE	-	-	-	-	-	-
user_average	FLOAT	NULLABLE	-	-	-	-	-	-
unique_products	INTEGER	NULLABLE	-	-	-	-	-	-

2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **평균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합**

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_data` AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE
      WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0
      ELSE ROUND(AVG(interval_), 2)
    END AS avg_interval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(
        InvoiceDate,
        LAG(InvoiceDate) OVER (PARTITION BY CustomerID ORDER BY InvoiceDate),
        DAY
      ) AS interval_
    FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
    WHERE CustomerID IS NOT NULL
  )
```

```
  GROUP BY CustomerID
)

SELECT
  u.*,
  pi.* EXCEPT (CustomerID)
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```



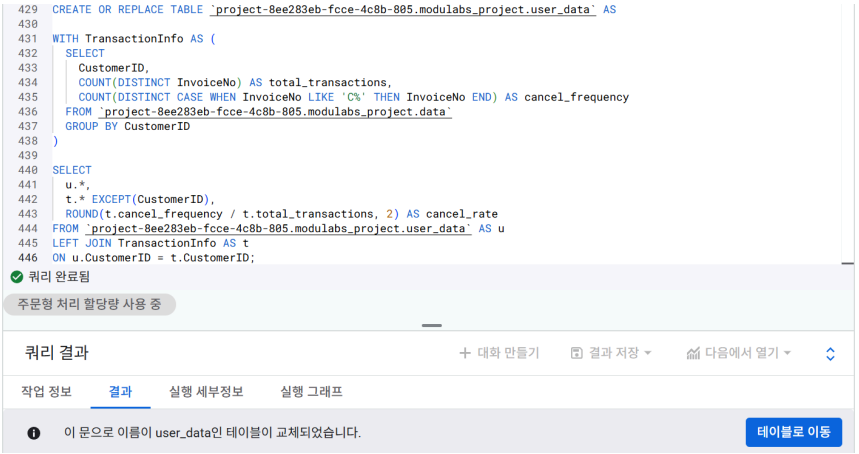
행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	avg_interval
1	16738	1	3	297	3.8	3.8	1	0.0
2	17102	1	2	261	25.5	25.5	1	0.0
3	16061	1	-1	269	-29.9	-29.9	1	0.0
4	13135	1	4300	196	3096.0	3096.0	1	0.0
5	18068	1	6	289	101.7	101.7	1	0.0
6	15389	1	400	172	500.0	500.0	1	0.0
7	17763	1	12	263	15.0	15.0	1	0.0
8	14576	1	12	372	35.4	35.4	1	0.0
9	14119	1	-2	354	-19.9	-19.9	1	0.0
10	15657	1	24	22	30.0	30.0	1	0.0
11	16323	1	50	196	207.5	207.5	1	0.0
12	13017	1	48	7	204.0	204.0	1	0.0
13	13120	1	12	238	30.6	30.6	1	0.0
14	12603	1	56	21	613.2	613.2	1	0.0
15	16428	1	-1	81	-3.0	-3.0	1	0.0
16	16953	1	10	30	20.8	20.8	1	0.0
17	13185	1	12	267	71.4	71.4	1	0.0
18	17948	1	144	147	358.6	358.6	1	0.0
19	13829	1	-12	359	-102.0	-102.0	1	0.0
20	17307	1	-144	365	-152.6	-152.6	1	0.0
21	14090	1	72	324	76.3	76.3	1	0.0
22	17291	1	72	308	550.8	550.8	1	0.0

페이지당 결과 수: 50 1 – 50 (전체 4362행)

3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
  - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
    (취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_data` AS

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency
  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.data`
  GROUP BY CustomerID
)

SELECT
  u.*,
  t.* EXCEPT(CustomerID),
  ROUND(t.cancel_frequency / t.total_transactions, 2) AS cancel_rate
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID;
```



- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
SELECT *
FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_data`;
```

```
448  SELECT *
449  FROM `project-8ee283eb-fcce-4c8b-805.modulabs_project.user_data`;
```

실행 시 이 쿼리가 374.86KB를 처리합니다.

주문형 처리 할당량 사용 중

쿼리 결과

+ 대화 만들기 | 결과 저장 | 다음에서 열기

작업 정보 | 결과 | 시각화 | JSON | 실행 세부정보 | 실행 그래프

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	avg_interval	total_transactions	cancel_frequency	cancel_rate
1	13938	1	70	35	71.7	71.7	7	0.0	1	0	0.0
2	16248	1	78	61	152.9	152.9	7	0.0	1	0	0.0
3	13227	1	99	201	257.7	257.7	9	0.0	1	0	0.0
4	18262	1	182	140	149.5	149.5	13	0.0	1	0	0.0
5	12355	1	240	214	459.4	459.4	13	0.0	1	0	0.0
6	16215	1	104	80	242.3	242.3	14	0.0	1	0	0.0
7	13886	1	126	71	243.6	243.6	16	0.0	1	0	0.0
8	14757	1	160	75	420.5	420.5	19	0.0	1	0	0.0
9	16627	1	162	79	322.8	322.8	20	0.0	1	0	0.0
10	15153	1	198	31	329.2	329.2	22	0.0	1	0	0.0
11	14184	1	127	66	452.9	452.9	24	0.0	1	0	0.0
12	14647	1	91	201	236.0	236.0	24	0.0	1	0	0.0
13	14489	1	299	207	463.4	463.4	30	0.0	1	0	0.0
14	15904	1	84	8	164.7	164.7	30	0.0	1	0	0.0
15	17972	1	36	241	135.2	135.2	30	0.0	1	0	0.0
16	15408	1	106	264	267.2	267.2	34	0.0	1	0	0.0
17	17251	1	108	362	269.8	269.8	37	0.0	1	0	0.0
18	13568	1	66	173	187.1	187.1	41	0.0	1	0	0.0
19	15074	1	431	105	735.7	735.7	41	0.0	1	0	0.0
20	15437	1	79	262	200.2	200.2	44	0.0	1	0	0.0

페이지당 결과 수: 50 1 – 50 (전체 4362행)

회고

Keep :

데이터분석 과정 중에 데이터를 정제하는 것을 빅쿼리로 익히기

불필요한 거래, 서비스 항목, 오류값을 제거하고 데이터를 분석 가능한 형태로 정리하는 방법

RFM(Recency, Frequency, Monetary) 지표와 추가 feature들을 SQL로 계산하면서 고객 구매 패턴을 분석하는 과정 이해하기

Problem :

SQL에서 집계 함수(MAX, COUNT 등)와 GROUP BY를 함께 사용할 때 오류가 발생하는 경우가 있었고,

날짜 처리나 윈도우 함수(LAG, DATE_DIFF 등)를 사용하는 과정에서 쿼리 구조를 이해하는 데 시간이 걸림

Try :

SQL의 복잡한 분석 쿼리도 빠르게 작성할 수 있도록 연습이 더욱 필요하다고 느꼈습니다.

RFM 분석 결과를 활용해 고객 세그먼트를 나누고 마케팅 전략까지 연결하는 분석도 해보고 싶습니다.